

OWNER'S IMPLEMENTATION GUIDE

From Custom GPT to Installable Plugin

A practical, approval-driven workflow for working with Codex

9 STAGES | OWNER DECISIONS | COPY-READY PROMPTS

Turn existing GPT instructions, knowledge, and workflows into focused, testable plugin skills—while retaining control of scope, privacy, licensing, user experience, and publication.

Includes a complete Cool Bible Tutor example

The workflow at a glance

You own the product decisions. Codex handles the technical execution. Progress through the stages in order, and approve each consequential decision before implementation or publication continues.

Stage	Owner's focus	Expected outcome
01	Inspect the source	Confirm the authoritative instructions, knowledge files, and open questions.
02	Define the scope	Approve the audience, supported workflows, exclusions, and architecture.
03	Set sharing boundaries	Decide which assets may be packaged, redistributed, or kept private.
04	Approve the skill map	Review the user-facing skills, routing, and requirement coverage.
05	Resolve behavior rules	Settle ambiguities and approve the tests that enforce each decision.
06	Build and validate	Authorize incremental implementation while retaining publication control.
07	Test the experience	Evaluate realistic requests, edge cases, and safety boundaries.
08	Choose distribution	Approve the release package and explicitly authorize any publication.
09	Manage future updates	Review changes, regression tests, rights, versions, and release notes.

NON-NEGOTIABLE CONTROL Codex must not publish externally, make rights decisions, broaden the approved scope, or expose private material without your explicit authorization.

How to use this guide

This guide is for the owner of a Custom GPT who wants Codex to convert existing instructions and reference materials into an installable plugin containing one or more focused skills. No programming, plugin-file design, or release-command experience is required.

Your job is to define the intended experience, make decisions that only the owner can authorize, test the result, and approve any external publication. Codex performs the source analysis, skill design, implementation, validation, packaging, and release preparation.

READING GUIDE Each stage explains your responsibility, the expected Codex output, copy-ready prompts, and the approval needed before moving forward.

What the finished plugin includes

At the end of this process, you should have:

- ☐ an installable plugin with one or more focused skills;
- ☐ the important behavior of the original GPT preserved in those skills;
- ☐ reference documents organized so Codex reads them only when relevant;
- ☐ tests showing which source requirements are covered;
- ☐ a validated plugin package;
- ☐ optional local, private GitHub, or public GitHub marketplace distribution;
- ☐ an owner-approved record of included, excluded, private, and licensed material.

The goal is not to copy the Custom GPT into a different folder. The goal is to preserve its useful behavior and knowledge in a structure that is easier to discover, test, maintain, and share.

Responsibilities: owner and Codex

You provide, decide, or approve	Codex analyzes, creates, or verifies
The authoritative source folder	An inventory of instructions and reference files
The intended users and outcomes	A proposed list of user goals and skills
Features to include or exclude	A skill architecture and routing design
Rights to redistribute documents and assets	A distribution boundary and rights checklist
Product behavior when the source is ambiguous	Clear, testable behavior rules
Plugin name, publisher, license, and visibility	Plugin manifests and marketplace metadata

You provide, decide, or approve	Codex analyzes, creates, or verifies
Representative real-world test requests	Behavioral tests and a coverage report
Approval of the user experience	Revisions and regression testing
Authorization to publish externally	Release packaging and GitHub publication

Decisions you may delegate to Codex

You can normally accept Codex's recommendation for:

- ☐ skill and reference filenames;
- ☐ folder organization;
- ☐ how detailed material is divided among skills and references;
- ☐ test implementation;
- ☐ validation tools;
- ☐ release packaging mechanics;
- ☐ safe handling of generated runtime files.

Example reply:

Use your recommended technical structure. Show me the skill names, their user-facing purposes, and anything that changes the original GPT experience before implementation.

Decisions Codex cannot make for you

Codex should not guess:

- ☐ whether a feature is in scope;
- ☐ whether a document may legally be redistributed;
- ☐ whether disputed source instructions should be changed;
- ☐ which publisher identity to use;
- ☐ which license you intend to grant;
- ☐ whether a GitHub repository should be public;
- ☐ whether Codex is authorized to push, tag, submit, or otherwise publish externally.

Prepare before you begin

Prepare one source folder containing the materials that define the Custom GPT.

Recommended source materials

- ☐ System instructions or an exported instruction document
- ☐ GPT name, description, and starter prompts
- ☐ Uploaded knowledge and reference documents
- ☐ Templates, examples, and images used by the GPT
- ☐ Existing scripts or tools, if any
- ☐ A resource index, if one exists
- ☐ Notes identifying obsolete or experimental files
- ☐ Information about ownership and redistribution rights

Do not reorganize the only copy of the source folder before Codex reviews it. Ask Codex to preserve the original files and perform conversion work in a separate plugin directory.

Information worth knowing in advance

You do not need every answer before starting, but Codex will probably ask about:

- ☐ intended users;
- ☐ primary language;
- ☐ required workflows;
- ☐ excluded features;
- ☐ teaching or response style;
- ☐ documents that may be redistributed;
- ☐ private or generated data;
- ☐ external services or live data;
- ☐ plugin and marketplace names;
- ☐ publisher identity and license;
- ☐ private versus public distribution.

STAGE 01 OF 09

Ask Codex to inspect the source folder

OWNER ACTION Identify the authoritative source materials and correct inventory mistakes.

APPROVAL GATE Approve the source inventory before Codex designs or builds anything.

What you do

Open the source folder as the Codex workspace and give Codex a conversion request. Tell it not to modify the original source documents and not to start implementation before you review the inventory and proposed skill structure.

Recommended starting prompt for this project

Review this source folder and help me convert the Custom GPT into a plugin of skills named cool-bible-tutor.

Do not modify the original system-instruction or reference documents.
Do not begin building the plugin yet.

First:

1. inventory the system instructions and reference files;
2. identify missing, duplicated, obsolete, or inconsistently named resources;
3. summarize the user goals and important behavior rules;
4. identify private, generated, or potentially restricted material;
5. list the decisions you need from me.

Ask consequential questions one at a time. After the questions, propose a skill structure for my approval.

Reusable prompt for another GPT

Review this source folder and help me convert the Custom GPT into an installable plugin of skills.

Preserve the original source files unchanged. First give me a source inventory, user-goal summary, risks, inconsistencies, and the decisions you need from me. Ask important questions one at a time.

Do not implement or publish anything until I approve the proposed skill structure and distribution boundary.

What Codex delivers

Codex should return a readable inventory containing:

- ☐ each source file and its apparent purpose;
- ☐ system instructions versus supporting knowledge;
- ☐ references named in instructions but missing from the folder;

- ☐ duplicate or inconsistent filenames;
- ☐ probable private or generated files;
- ☐ probable rights or attribution questions;
- ☐ an initial list of user goals;
- ☐ questions requiring owner decisions.

What you review

Check whether Codex found the authoritative files. Correct anything that is outdated, mislabeled, or misunderstood.

Example clarification

Codex might ask:

The resource index mentions `identify_relation_guide.md`, but the folder contains `identify_relationships_guide.md`. Are these the same resource?

You might reply:

Yes. Treat them as the same resource. Use `relationships.md` as the canonical plugin filename, but preserve the original source document.

APPROVAL GATE

Do not approve skill design until the inventory contains every authoritative instruction and reference source.

Example approval:

The source inventory is complete. Use these files as authoritative. Proceed to ask me the scope and rights questions, but do not build the plugin yet.

STAGE 02 OF 09

Decide the plugin's scope

OWNER ACTION Define the audience, required capabilities, exclusions, and preferred experience.

APPROVAL GATE Approve the functional scope and any external-service dependency.

What you do

Tell Codex which original capabilities must remain, which should be excluded, and what kind of experience users should receive.

Questions Codex may ask

- ☐ Who will use this plugin?
- ☐ What are the three most important things users should be able to do?
- ☐ Which GPT modules are required?
- ☐ Which modules are obsolete or outside this release?
- ☐ Should the plugin reproduce every starter prompt?
- ☐ What language should it use by default?
- ☐ Should it guide users interactively or give complete answers immediately?
- ☐ Does it require current external data or account access?

Example decisions from Cool Bible Tutor

The primary users are Traditional Chinese-speaking Christians who want guided Bible study, theological discussion, or Scripture-shaped life application.

Include the ten-stage inductive Bible-study workflow, theological discussion, application, exegetical context, word and translation comparison, and verified Chinese Union Version retrieval.

Exclude the church-ministry prompt-template module. It is outside the scope of this plugin release.

Use Traditional Chinese by default. Short English or source-language terms may be included when they improve understanding.

When to choose skills only

Tell Codex to prefer a skills-only plugin when the GPT relies on instructions, packaged references, templates, and local scripts.

Example:

Prefer a skills-only plugin. Do not add an MCP server unless you find a workflow that genuinely requires live remote data, authentication, or controlled external actions. Explain the need before proposing one.

Codex can recommend whether an MCP server is necessary, but you should approve any new external service dependency because it changes setup, privacy, authentication, and publication requirements.

What Codex delivers

- ☐ an in-scope capability list;
- ☐ an explicit exclusion list;
- ☐ intended users and language;
- ☐ recommended skills-only or MCP-backed architecture;
- ☐ unresolved product decisions.

APPROVAL GATE

Example:

I approve this scope and the skills-only recommendation. Preserve the explicit exclusions in the project coverage document. Proceed to the rights and distribution review.

STAGE 03 OF 09

Decide what may be packaged and shared

OWNER ACTION Decide whether every source asset may be packaged, shared, or redistributed.

APPROVAL GATE Approve an explicit distribution boundary before implementation proceeds.

What you do

For each reference, PDF, database, image, template, model, script, or third-party component, tell Codex whether it may be included in a public package.

Uploading a file to a Custom GPT does not by itself prove that it may be redistributed in a GitHub repository.

Questions Codex may ask

- ☐ Do you own this document?
- ☐ Is there a license or written permission to redistribute it?
- ☐ Is attribution required?
- ☐ May the file be modified?
- ☐ Does it contain private, pastoral, personal, or user-generated information?
- ☐ Is it a source document or generated output?
- ☐ Should ordinary users receive it, or should each user supply their own copy?
- ☐ Does the package risk implying endorsement by the original source?

Example replies

When distribution is authorized:

I authorize redistribution of these two specified Bible PDFs in this plugin. Record their filenames, provenance, and SHA-256 hashes. Include an attribution notice and do not imply endorsement by the source.

When distribution is not authorized:

Do not package this commentary PDF. Document how an authorized user can supply it locally, and ensure the plugin still behaves safely when the file is absent.

For generated or private data:

Keep OCR output, review history, authoring databases, backups, user corrections, logs, and private vector data outside the plugin and outside Git. Package only explicitly approved immutable runtime assets.

If you do not know the rights status, say so:

The redistribution rights are unclear. Exclude the file from the public release until I confirm them. Do not infer permission.

What Codex delivers

- ☐ an included-assets list;
- ☐ an excluded-assets list;
- ☐ required attribution and third-party notices;
- ☐ a plan for private and generated data;
- ☐ a proposed release boundary that can be tested automatically.

APPROVAL GATE

Example:

I approve this distribution boundary. Treat excluded files and private runtime data as release blockers if they appear in the package. Proceed to propose the skill structure.

STAGE 04 OF 09

Review and approve the skill map

OWNER ACTION Review each proposed skill as a recognizable, useful user outcome.

APPROVAL GATE Approve the skill architecture, routing, and requirement coverage.

What you do

Review the proposed skills as a product owner. Concentrate on what each skill helps the user accomplish, not its internal files.

What a useful proposal looks like

Proposed skill	User goal	Main responsibility
Guide Bible Tutor sessions	Start or continue a guided study	Select the mode and coordinate the learner's progress
Observe biblical passages	Discover what a passage says	Guide textual observation before interpretation
Interpret biblical passages	Understand the passage's meaning	Answer questions, synthesize meaning, and identify a theme
Apply biblical truth	Put established meaning into practice	Form principles, concrete actions, and follow-through
Discuss biblical theology	Explore a theological question	Guide Scripture-grounded, fair, Socratic discussion
Support biblical exegesis	Examine a passage in context	Select the most relevant interpretive contexts
Compare words and translations	Understand a material language difference	Explain verified lexical, grammatical, and translation evidence
Retrieve Chinese Union Version Scripture	Obtain exact passage text	Retrieve verified text and report provenance or uncertainty

Questions to ask yourself

- ☐ Does every skill describe a recognizable user goal?
- ☐ Are two proposed skills really one workflow?
- ☐ Is one proposed skill trying to do too much?
- ☐ Does the coordinating skill duplicate the other skills?
- ☐ Can users understand the skill names?

- ☐ Is any important GPT capability missing?
- ☐ Is an excluded feature accidentally included?

Example requested revision

Keep the eight proposed skills, but do not expose RAG as a ninth user-facing skill. Treat semantic discovery as supporting retrieval technology. Exact quotation must still use the verified Scripture retrieval workflow.

What Codex delivers

- ☐ the approved skill map;
- ☐ how the coordinating skill routes among other skills;
- ☐ a requirement coverage matrix mapping source instructions to skills;
- ☐ behavior scenarios that will prove the mapping works.

APPROVAL GATE

I approve this eight-skill structure and routing model.

Before implementation, show me the important ambiguous behavior decisions and the representative tests you plan to run.

STAGE 05 OF 09

Resolve behavior decisions

OWNER ACTION Resolve contradictory instructions and make the intended experience testable.

APPROVAL GATE Approve the behavior decision log and its representative tests.

What you do

Resolve tensions or ambiguities in the old GPT instructions. These decisions affect the experience more than folder names or manifest fields.

Typical behavior decisions

Decision	Possible choices
Guided versus direct answers	Always guided; direct when explicitly requested; always direct
Progression	Automatic; ask before each major stage; user-selectable
Missing information	Ask; provide a limited answer; refuse
Source uncertainty	State uncertainty; use model memory; stop until verified
Denominational differences	Choose one view; compare recognized views fairly
Default language	Original GPT language; user's language; configurable
Detail level	Beginner; advanced; adapt to the user

Example Codex question and user reply

Codex might ask:

The instructions say both “avoid giving answers” and “answer general questions directly.” What should happen when a learner explicitly requests a direct explanation?

Recommended reply:

Use guided learning by default. When the user explicitly requests a direct answer, give a concise explanation or one worked example, then offer to return to guided study. Do not force the full tutorial.

Another question might be:

May semantic retrieval results be quoted as exact Scripture?

Reply:

No. Semantic retrieval may suggest candidate references only. Exact Scripture quotation must come from the verified passage-retrieval path. State clearly when verification is unavailable.

Ask Codex to keep a decision log

Record every approved behavior decision in a short decision table. For each decision, show the source tension, the approved rule, the owning skill, and the test that verifies it.

What Codex delivers

- ☐ a concise behavior decision log;
- ☐ planned tests for each important rule;
- ☐ identified stop, wait, consent, or safety conditions;
- ☐ any remaining decisions only the owner can answer.

APPROVAL GATE

I approve the behavior decision log and proposed tests. Build the plugin in small, reviewable stages. Preserve the original source files.

STAGE 06 OF 09

Let Codex build and validate the plugin

OWNER ACTION Review incremental progress and flag deviations from approved behavior.

APPROVAL GATE Authorize implementation only; publication remains prohibited.

What you do

You do not need to direct every implementation detail. Ask Codex to report outcomes in language you can review.

Recommended build prompt

Build the approved plugin.

Work in small, reviewable stages. After each stage, report:

1. the user-facing capability completed;
2. the source requirements it covers;
3. the tests and validation that passed;
4. any deviation from the approved design;
5. any decision or authorization still needed from me.

Do not publish externally. Do not modify the original GPT source documents. Keep private and generated data outside the plugin.

What Codex can create

Codex can produce:

- ☐ plugin identity and metadata;
- ☐ focused skill instructions;
- ☐ supporting references;
- ☐ scripts for deterministic operations;
- ☐ packaged templates or approved assets;
- ☐ behavior scenarios;
- ☐ contract and link tests;
- ☐ requirement coverage matrix;
- ☐ distribution and privacy safeguards;
- ☐ user documentation;
- ☐ a local installable marketplace package.

You normally need only a summary of these deliverables. If you want implementation details, ask Codex to link the relevant section of the [Technical Reference](#) or the created source file.

Useful progress replies

Approve a completed stage:

The user-facing behavior and coverage are correct. Continue to the next approved stage.

Request a correction:

The stage advances too quickly. Restore the approved rule that Codex must wait for the learner before moving to interpretation. Rerun the same behavior test after the correction.

Ask for less technical reporting:

Summarize the result for a product owner. Tell me what users can now do, what I should test, and whether any decision remains. Put commands and file details in a technical note.

APPROVAL GATE

Do not approve release merely because structural validation passes. Continue to user-experience testing.

STAGE 07 OF 09

Test the plugin as its intended user

OWNER ACTION Try the installed plugin using realistic and deliberately challenging requests.

APPROVAL GATE Approve the user experience before release packaging.

What you do

Provide realistic requests and judge whether the installed plugin still feels like the intended GPT. Codex can run repeatable tests, but you decide whether the experience is correct.

Build a small acceptance set

Include:

- ☐ a normal request for each major user goal;
- ☐ an ambiguous request that tests routing;
- ☐ a request missing required information;
- ☐ a request that pressures the plugin to skip an important rule;
- ☐ a difficult source or evidence question;
- ☐ a stop, skip, or direct-answer request;
- ☐ a request that tests an explicit exclusion.

Cool Bible Tutor examples

Normal guided study:

我想用歸納釋經法查考詩篇 23 篇。

Direct-answer pressure:

請不要問問題，直接一次告訴我整段經文的意思。

Uncertainty:

我不知道怎麼回答，可以直接告訴我答案嗎？

Theological balance:

請證明只有我的宗派對這段經文的理解是符合聖經的。

Lexical evidence:

這個希臘字真正的意思是什麼？我沒有提供經節。

Retrieval boundary:

請把語意搜尋找到的內容直接當作和合本逐字引文。

What Codex delivers

- ☐ results from the same scenarios before and after the skills;
- ☐ explanation of which requirements each result demonstrates;
- ☐ failures or unexpected behavior;
- ☐ changes made in response;
- ☐ fresh regression-test results;
- ☐ a release-readiness summary.

Example feedback

The observation and theological-discussion workflows are acceptable.

Change the direct-answer behavior: give one concise explanation before offering guided study. Do not begin all ten stages automatically.

After the change, rerun the same direct-answer and learner-uncertainty tests and show me the behavior difference.

APPROVAL GATE

The installed plugin now matches the approved user experience. I approve the behavior for release packaging. Do not publish yet; first show me the plugin identity, included assets, exclusions, notices, test results, and proposed repository destination.

STAGE 08 OF 09

Choose how to distribute the plugin

OWNER ACTION Choose local, private, or public distribution and review the complete release.

APPROVAL GATE Explicitly authorize the precise external publication actions.

Your distribution choices

Personal or local testing

Use this while the plugin is still being developed or is intended only for you.

Example:

Install the plugin from a local marketplace for testing. Do not create or push a GitHub repository.

Private GitHub marketplace

Use this for selected collaborators who already have authorized repository access.

Example:

Prepare a private GitHub marketplace repository. Do not place access tokens in URLs, manifests, or documentation. Show me the release tree and proposed repository settings before publishing.

Public GitHub marketplace

Use this when the plugin and all packaged assets are approved for public redistribution.

Example:

Prepare this as a public GitHub marketplace repository named obvious-one-plugins.

Use:

- marketplace name: obvious-one
- plugin name: cool-bible-tutor
- publisher: schao523
- license: MIT
- default branch: main

Show me the final package, distribution audit, Git diff, installation instructions, and release tag before pushing. Do not publish until I explicitly approve.

Decisions Codex will need

- ☐ plugin display name and stable identifier;
- ☐ version number;
- ☐ publisher name and public contact information;
- ☐ license;
- ☐ repository owner and name;

- ☐ marketplace name and display name;
- ☐ public or private visibility;
- ☐ approved descriptions and starter prompts;
- ☐ privacy, security, support, and terms URLs when applicable;
- ☐ whether this is only GitHub distribution or also intended for OpenAI review.

Understand the two publication paths

A GitHub repository marketplace is a distribution source that users can add to Codex. Publishing that repository does not automatically list the plugin in the universal plugin directory shared by ChatGPT and Codex.

Universal-directory publication requires a separate OpenAI Platform submission, verified publisher identity, listing materials, policy attestations, and review. Ask Codex to consult the current [official OpenAI plugin submission documentation](#) when you are ready for that separate process.

What Codex delivers before publication

- ☐ final plugin name and version;
- ☐ repository and marketplace identity;
- ☐ skill list and user-facing purpose;
- ☐ included and excluded assets;
- ☐ licensing and third-party notices;
- ☐ test and validation results;
- ☐ distribution-audit result;
- ☐ release manifest or file summary;
- ☐ proposed Git changes;
- ☐ installation and upgrade instructions;
- ☐ any remaining risk or unresolved decision.

FINAL PUBLICATION AUTHORIZATION

Only send this after reviewing the release summary:

I approve the displayed release contents, plugin version, marketplace metadata, notices, and public GitHub destination.

Publish the marketplace repository, create the approved version tag, then install the plugin from the GitHub marketplace in a new Codex task and rerun the release smoke tests.

If you want Codex to prepare everything but not push:

Prepare the final repository and release commands, but do not push, create a tag, submit to OpenAI, or make any other external change. I will publish it myself.

STAGE 09 OF 09

Review updates after publication

OWNER ACTION Review the impact of later changes on behavior, rights, tests, and versioning.

APPROVAL GATE Approve each update and its release separately.

For later changes, begin with the existing plugin and the source change that motivates the update.

Recommended update prompt

Review the requested change against the current plugin, its source requirements, behavior tests, and distribution boundary.

Before editing, tell me:

1. which skills or assets are affected;
2. whether the user experience changes;
3. which tests must be updated or added;
4. whether the version should change;
5. whether any new rights, privacy, or publication decision is needed.

What you review

- ☐ whether the change matches the original purpose;
- ☐ whether it silently broadens the plugin scope;
- ☐ whether existing users will see changed behavior;
- ☐ whether a new asset can be redistributed;
- ☐ whether release notes are understandable;
- ☐ whether the marketplace-installed version was retested.

Update approval example

I approve this backward-compatible behavior improvement as a minor release. Preserve all existing distribution exclusions. Implement it, rerun affected scenarios and full validation, then show me the release diff before publishing.

Appendix A — owner approval checklists

Before approving implementation, confirm:

- ☐ Codex identified the authoritative source instructions.
- ☐ Missing and inconsistent references were resolved.
- ☐ Intended users, language, and primary outcomes are clear.
- ☐ Required and excluded capabilities are explicit.
- ☐ Skills-only versus MCP architecture is approved.
- ☐ Every packaged asset has a distribution decision.
- ☐ Private and generated data will remain outside the plugin.
- ☐ The proposed skill map reflects recognizable user goals.
- ☐ Important behavior ambiguities have approved answers.
- ☐ Planned tests represent real user requests.

Before approving publication, confirm:

- ☐ You tested the installed plugin as its intended user.
- ☐ Important behavior and pressure scenarios pass.
- ☐ Plugin name, version, publisher, and license are correct.
- ☐ Included and excluded assets match your decision.
- ☐ Notices and attribution are complete.
- ☐ No private data, credential, cache, or local path is packaged.
- ☐ The GitHub repository visibility is correct.
- ☐ You reviewed the proposed release contents and Git changes.
- ☐ GitHub marketplace publication and OpenAI submission are not being confused.
- ☐ External publication has your explicit authorization.

Appendix B — reusable owner prompts

Ask Codex to explain a decision simply

Explain this decision for a nontechnical product owner. Give me your recommended choice, the user-visible effect, the main tradeoff, and the exact reply I can send if I accept it.

Ask Codex to distinguish owner decisions from technical choices

Separate the remaining questions into:

1. decisions only I can make;
2. technical choices you can make using your recommendation;
3. actions that require my approval immediately before execution.

Ask for a concise progress report

Report progress in five parts: completed user capability, source coverage, tests, remaining owner decision, and next approval point. Do not include implementation details unless a failure requires them.

Ask for a user-facing release review

Prepare a release review for the plugin owner. Summarize what users can do, included and excluded content, test evidence, privacy and rights boundaries, repository destination, and the exact external actions you will take after approval.

Stop external publication

Continue local implementation and testing, but do not push, publish, submit, create releases, create tags, or change external repositories until I explicitly authorize those actions.

Appendix C — final handover requirements

You should receive a concise completion summary containing:

- ☐ where the plugin and marketplace files are located;
- ☐ the user-facing skills and what each one does;
- ☐ source requirements and exclusions covered;
- ☐ behavior and validation results;
- ☐ included and excluded assets;
- ☐ privacy, rights, and external-data boundaries;
- ☐ current version and publication status;
- ☐ installation and testing instructions;
- ☐ remaining decisions or known limitations.

You do not need a long explanation of every internal file unless you request it. The [Plugin Skills Technical Reference](#) remains available for maintainers who need the detailed implementation and release procedures.